Encoder Scaling Cross Check

Zachary Stone

October 19, 2026

Outline

- Problem Statement
- Problem Explanation
- Solution Strategy
- Implementation Details
- Measurement Error
- Filtering Specifications
- Trial Solution
- Production Solution
- Solution Verification

Problem Statement

Situation

- Bringing up new hardware system

Observed Problem

- Car is overshooting the landing or braking early

Observed Design Flaw

- Discovered the controller and drive have mismatched rotational scale factors

Problem Explanation

Two components using the same rotational encoder.

1. Drive computes velocity.
2. Controller computes position.

Position feeds distance to target, controls braking signal.

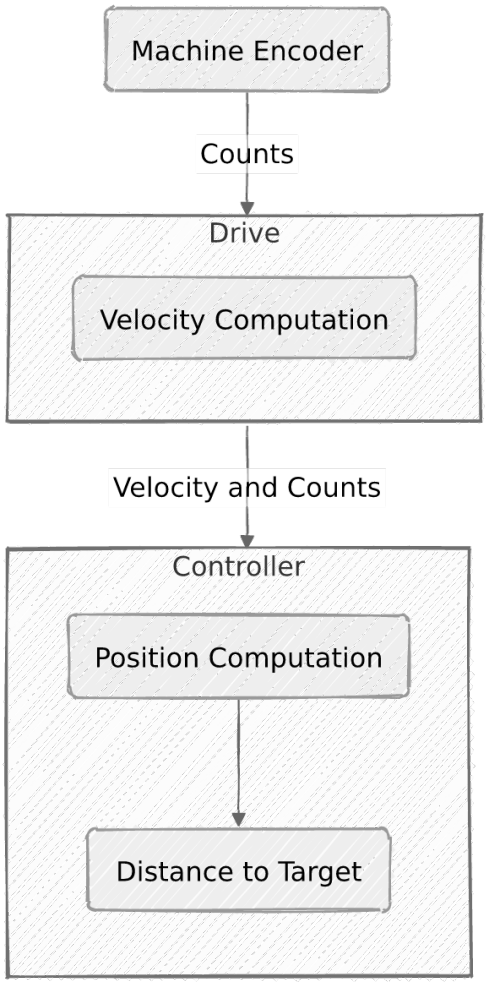
Mismatch of scale factors converting rotational counts to linear distance means:

$$V \neq \frac{dp}{dt}$$

Calculated distance to target does not reflect the real world.

Car overshoots landing or has lengthy deceleration and creep to stop.

Machine Encoder Information Flow



Solution Strategy

Cross check the drive velocity with the rate of change of controller position.

Early detection allows us to stop the car and return to the prior landing slowly and remove from service.

Expected Effect

We expect the error signal V_E is proportional to the drive velocity V_D as well as the ratio of the controller scale factor S_C to the drive scale factor S_D.

$$V_E = V_D(1 - \frac{S_C}{S_D})$$

If we can detect a 10% velocity error, we can detect a 10% scaler difference.

Solution:

$$V_C = \frac{\Delta C_E * S_C}{\Delta t}$$

$$V_D = \frac{\Delta C_E * S_D}{\Delta t}$$

$$\frac{V_D}{V_C} = \frac{S_D}{S_C}$$

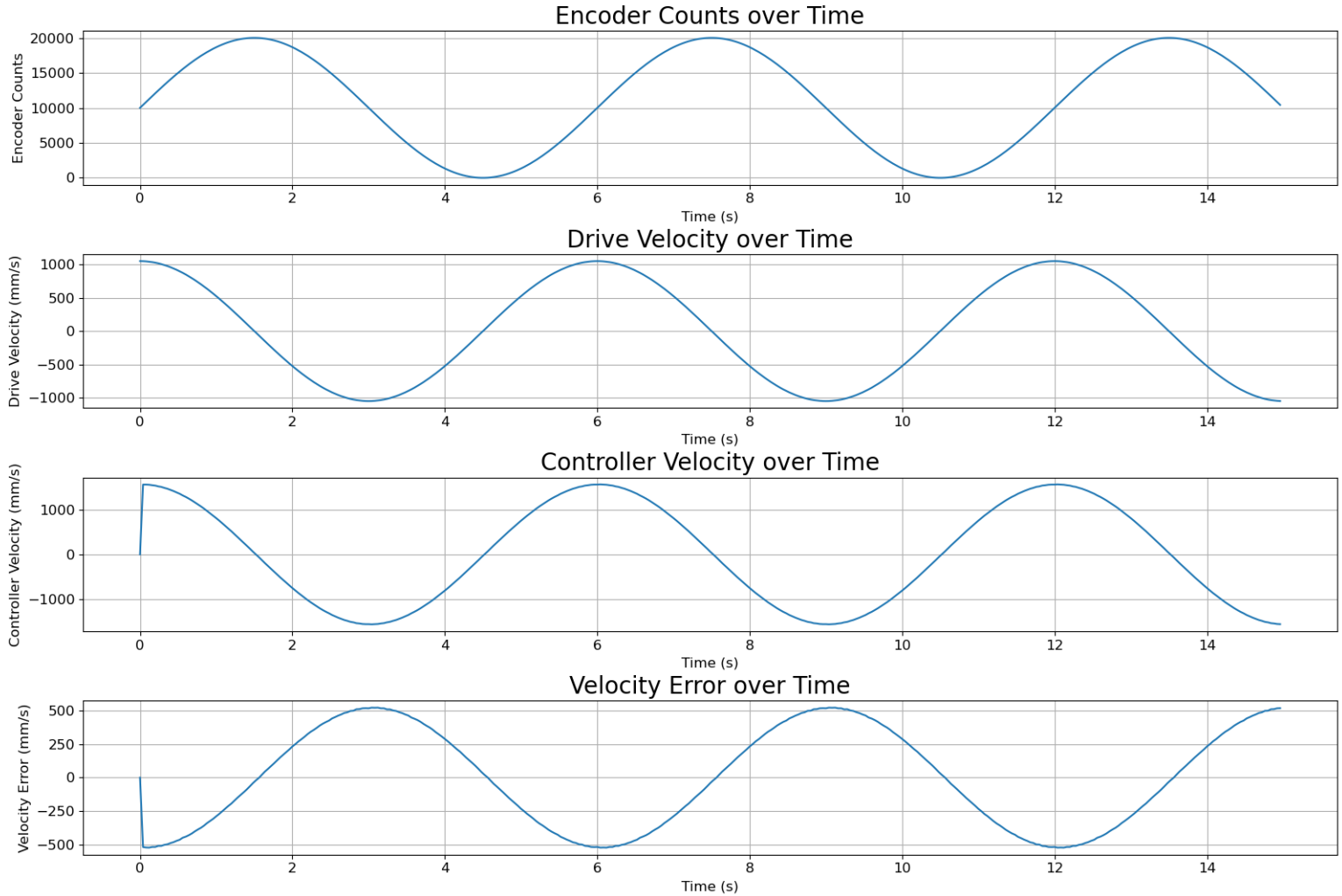
$$V_C = V_D(\frac{S_C}{S_D})$$

$$V_E = V_D - V_D(\frac{S_C}{S_D})$$

C_E is encoder counts, t is time.

Expected Effect Example

Python helps me validate my process. Here, the controller velocity is being computed from encoder counts with with a 50% difference in scalers.

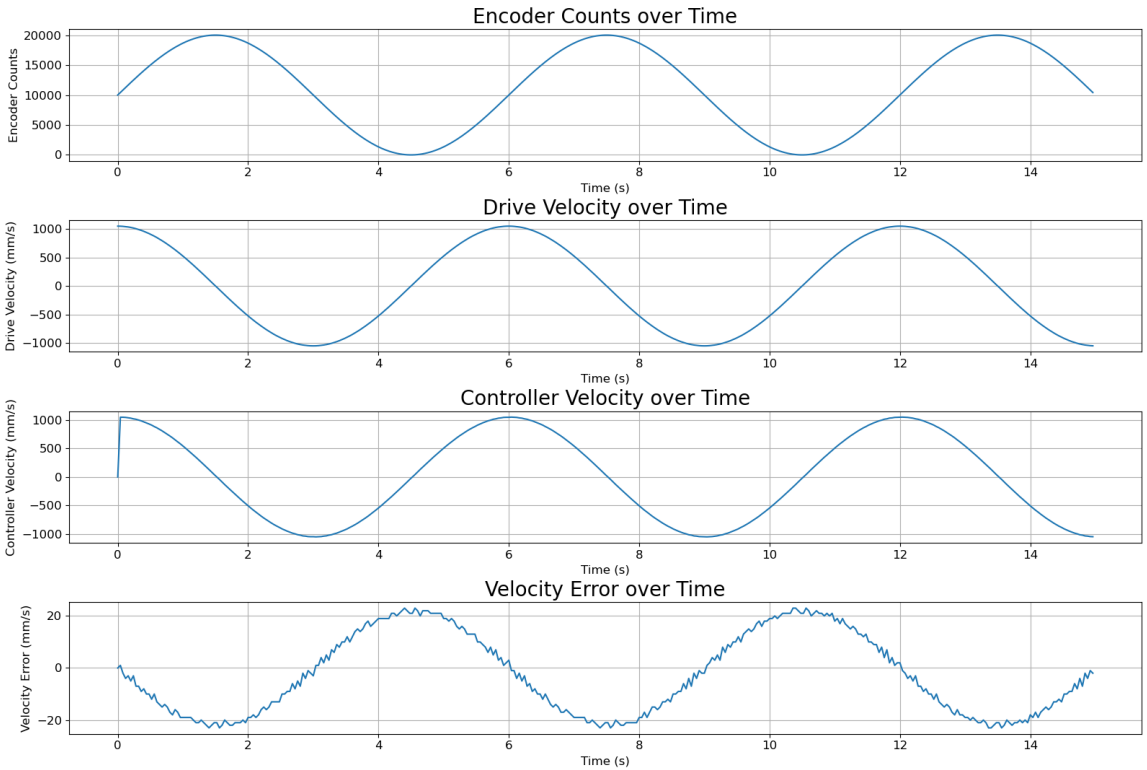


Implementation Plan

1. Compute velocity from encoder counts
2. Filter out noise
3. Compare with drive velocity

Measurement Error

Methods of computation and filtering can introduce error where there is none.



Features:

- Proportional to acceleration
- Negative sine
- Zero crossings

Source:
Comparing an average (constant) velocity for one cycle to an instantaneous measurement for the end of the same cycle.

Measurement Error Explained

Magnitude: We can show the magnitude of the error is about half a cycle worth of acceleration. Below, a is acceleration at a point in time and Δt is one cycle of time.

$$V_E \approx \frac{1}{2} a \Delta t$$

Controller Velocity:

$$V_C = \frac{P_1 - P_0}{\Delta t}$$

$$P_1 = P_0 + V_0 \Delta t + \frac{1}{2} a \Delta t^2 + \frac{1}{6} j \Delta t^3$$

$$V_C = V_0 + \frac{1}{2} a \Delta t + \frac{1}{6} j \Delta t^2$$

Drive Velocity:

$$V_D = V_0 + a \Delta t + \frac{1}{2} j \Delta t^2$$

Impact:

If we can't eliminate this measurement error, it is going to set a lower limit on the scaler mismatch we can detect.

Filtering Specifications

- Moving average filters will be applied for disturbance rejection.
- Filter drive velocity and controller velocity.
- Eliminate measurement error in the filtering.

Drive velocity receives a moving average filter with size 5.
Controller velocity receives a moving average filter of size 4.

$$V_{avg} = \frac{v_0 + v_1 + ... + v_{N-1}}{N}$$

If we experience a high frequency disturbance of 20% the expected value, this is attenuated by $\frac{1}{N}$

Filtering Specifications Continued

The choice of mismatched filter sizes seems odd, but actually reduces our measurement error by slightly increasing the phase delay of the drive velocity.

Consider window size 1 for controller velocity and window size 2 for drive velocity.

Average Controller Vel

$$V_C = V_0 + \frac{1}{2}a\Delta t + \frac{1}{6}j\Delta t^2$$

Average Drive Vel

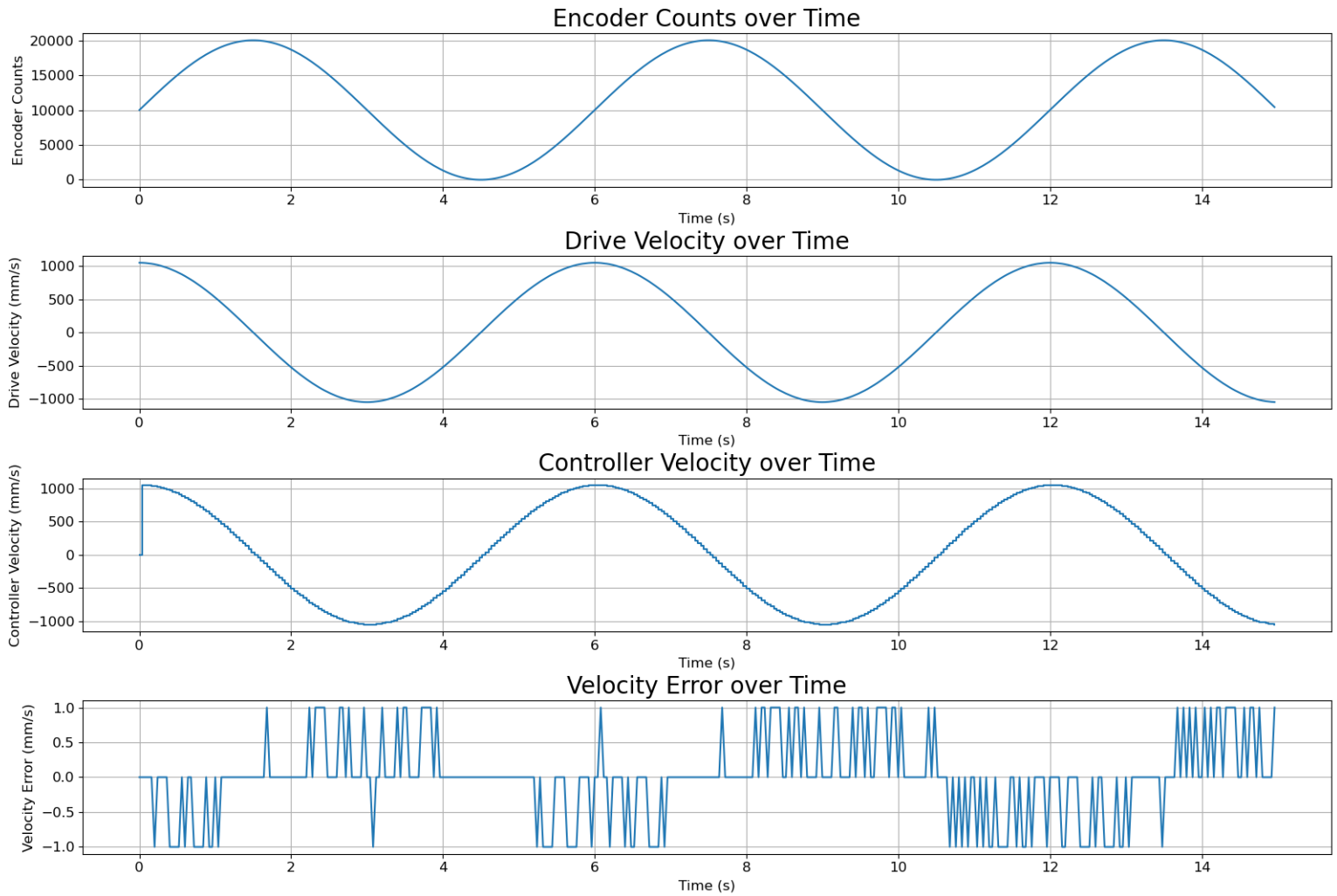
$$V_D = \frac{V_{D0} + V_{D1}}{2}$$

$$V_D = \frac{2V_{D0} + a\Delta t + \frac{1}{2}j\Delta t^2}{2}$$

$$V_D = V_{D0} + \frac{1}{2}a\Delta t + \frac{1}{4}j\Delta t^2$$

Filter Implemented

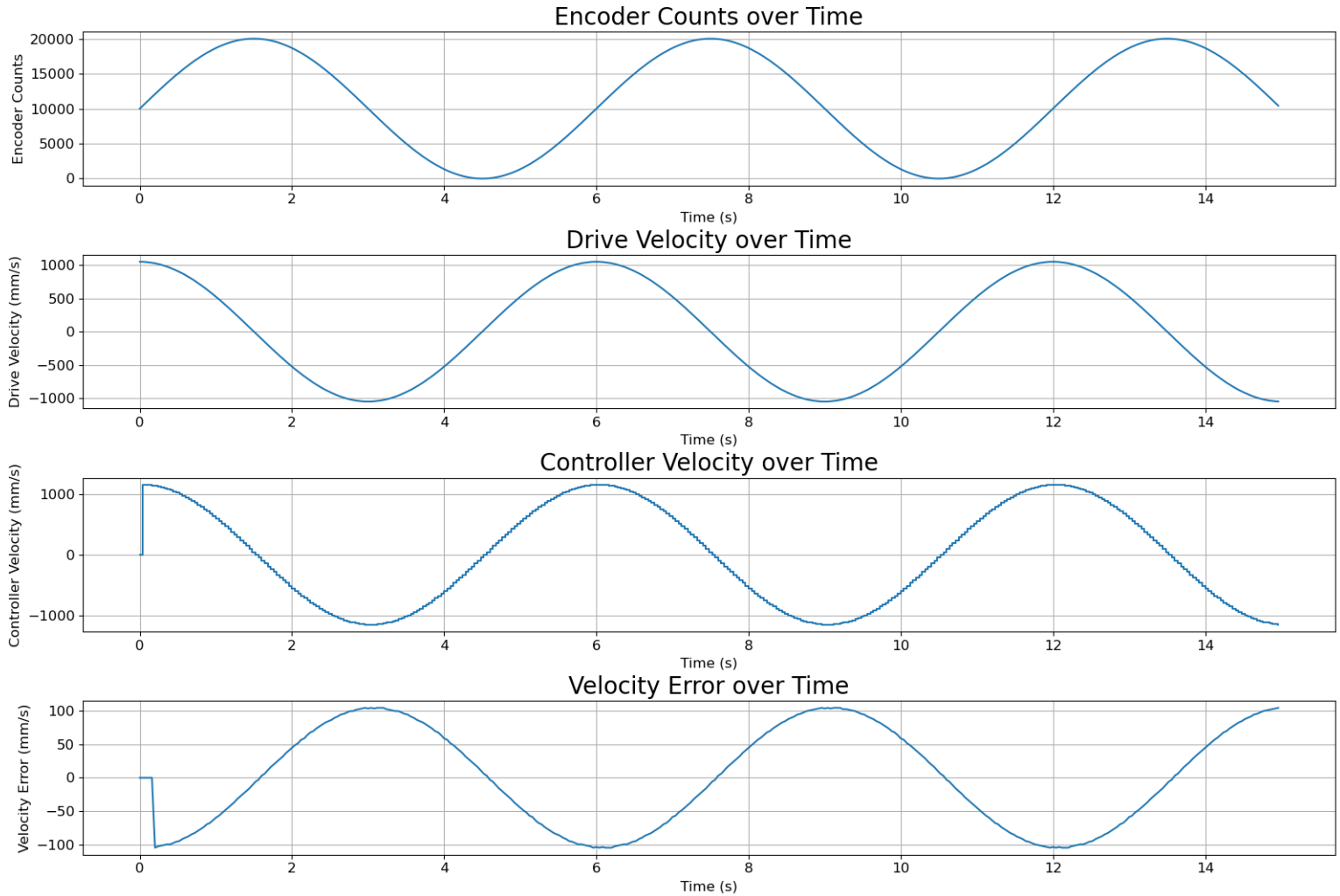
Here the moving average filters are implemented as described above, with no scaler mismatch.



Trial Solution from CAN Data

CAN data trace taken and parsed with Python. Solution was trialed in Python script.

Applied 10% difference in scalers and the filtering described above.



Real Solution

Implementation in C is broadly the same, with a few adaptations.

- Handle encoder wrap around
- Handle timestamp wrap around
- Consider scenarios in which the data is unreliable (communication issues)

Verification

- Implemented outputs in the solution so velocity error is accessible over CAN.
- Configured the system correctly and observed minimal error.
- Configured the system incorrectly and observed the expected error.
- Controller detects mismatch during initial acceleration, stops and returns to initial landing.